Python 0/1 → Web

One-page learning map · 2-day high-scope facilitation
From how machines store bits to building a web API

DAY 1 · Phases 1–4 | DAY 2 · Phases 5–8 | PARALLEL TRACKS

0 1 Machine world — Binary · bits & bytes
DECODING ◂ ▸ ENCODING
Python — Interpreter · high-level · slower than compiled, faster to write
DECODING ◂ ▸ ENCODING
Humans — Language · decimal
Quantum — qubits · 0 and 1 at once (next frontier)

EVERYTHING HUMANS MAKE BECOMES BYTES: Numbers · Alphabets · Special characters · Images · Videos · Audio
READ / WRITE: `P1 ⇄ P2` programs talk through files, databases & HTTP

1 Foundations
DAY 1 · WHAT PYTHON IS

- **Definition:** high-level, interpreted, general-purpose language
- **Runtime:** source `.py` → bytecode → Python Virtual Machine (CPython)
- **Memory:** objects live on the **heap (RAM)**; names only point to them
- **Interpreted** line-by-line: portable, slower than compiled

Use it for
AI/ML, data, web, automation, scripting

Avoid for
low-level systems, 3D engines, CPU-heavy threads

2 Core Basics
DAY 1 · NAMING PLACES IN MEMORY

IDENTIFIER = NAME OF A PLACE

Variables
value can change
`count = 0`

Constants
by convention, fixed
`MAX_SIZE = 10`

NAMING CONVENTIONS (PEP 8)

`snake_case` vars, functions
`PascalCase` classes
`UPPER_CASE` constants
`camelCase` JS / Java

- letters, digits, _; no leading digit; case-sensitive; no keywords
- # comments, `"""docstrings"""`; first I/O `print()` `input()`

3 Data Types, Sequences & Casting
DAY 1 · HOW PYTHON STORES WHAT HUMANS MEAN

GROUP	TYPES	NATURE
Data types single values	`int` `float` `complex` `bool`	IMMUTABLE
Data type + sequence changeable containers	`list` seq · 1D / multi-D `set` set `dict` mapping `bytearray` 2D	MUTABLE
Data type + sequence fixed containers	`tuple` seq `frozenset` set `range(0,10,2)` seq `str` seq `bytes`	IMMUTABLE
Special no value / views	`NoneType` `memoryview` M / I	IMMUTABLE view follows source

Working with sequences (str · list · tuple · range)

`s[0]`	first item	`s[-1]`	last item
`s[1:4]`	slice 1 → 3	`s[::-1]`	reverse
`len(s)`	length	`x in s`	membership

Index from **0**; a mutable list changes in place, an immutable str/tuple returns a new object.

Media as bytes

`bytearray`	2D grid (e.g. grayscale)
`bytes (I)`	Image → 3D: H × W × RGB
`bytes (V)`	Video → 4D: frames × H × W × RGB
`audio`	samples over time

Built-in type casting
`int()` `float()` `str()` `bool()` `list()` `tuple()` `set()` `dict()`
`int("42") → 42 · bool("") → False`

3+ Relationships · Operators
DAY 1 · COMBINING VALUES INTO EXPRESSIONS

ARITHMETIC	ASSIGNMENT	RELATIONAL	LOGICAL	BITWISE	MEMBERSHIP	IDENTITY	→ Expression / Condition
`+ - * /` `// % **`	`= += -= *=` `//= **= :=`	`== != < >` `<= >=`	`and or not`	`& | ^ ~` `<< >>`	`in` `not in`	`is` `is not`	evaluates to a value or True / False

4 Control Flow & Flow Disruptors
DAY 1 · DECIDING & REPEATING

Decide
```
if / elif … / else
match cmd: case …:
case _:
```

Repeat
```
for x in seq:
while condition:
```

FLOW DISRUPTORS

`break`	exit loop · `continue` next turn · `pass` do nothing
`return`	**finish** — end function, send value
`yield`	**smooth** — pause & resume (generator)

USER EXCEPTIONS

`try`	D Do the risky work
`except`	E runs if an Error occurs
`else`	NE runs if No Error
`finally`	C always runs — Clean-up

5 Sub-programs · Functions
DAY 2 · REUSABLE BLOCKS OF LOGIC

System-defined
```
input() print()
open() close()
read() write()
len() type()
```

User-defined
```
def area(r):
  return 3.14*r*r
```
scope: local / global

Lambda
```
lambda x: x*2
```
with `enumerate` `zip` `map` `filter`

PARAMETERS (IN DEF) · ARGUMENTS (IN CALL)

`1 · a`	Standard positional
`2 · a=1`	Default
`3 · *args`	Arbitrary positional → tuple
`4 · **kw`	Arbitrary keyword → dict

```
# * and ** are mirror images
def f(*args, **kwargs): ...  # pack
f(*[1, 2], **{"x": 3})     # unpack
```

Dunder / magic (double-underscore): `__init__` `__str__` `__len__` → OOP

6 Modules, Packages & Library
DAY 2 · ORGANISING CODE

Module = one Python file
```
import math
from os import path
```

Package = multiple modules
folder + `__init__.py`
`pip install requests` · venv

BUILT-IN STANDARD LIBRARY: `os` `math` `datetime` `json` `random`

7 Object-Oriented Programming
DAY 2 · MODELLING REAL THINGS

Class → Object
blueprint → instance; constructor `__init__`, `self`

Pillars
Inheritance (super) · Polymorphism · Encapsulation (_, __) · Abstraction

8 Working with the Web
DAY 2 · PYTHON TALKS TO THE WORLD

HTTP
request → response · GET / POST · status codes · JSON

FastAPI
```
@app.get("/items")
```
run with uvicorn

Running in parallel ▸ every phase
Hatched on the board = woven through Days 1 & 2, not a separate session

TRACK A Testing & Evaluation
Syntax error · Runtime error · Logical error · Boundary error
assert → Unit tests → Integration tests

P2	read tracebacks; spot syntax errors
P3	predict types & casting results
P4	edge & boundary inputs; try/except
P5–7	unit-test each function & class
P8	integration-test the API endpoints

TRACK B Input / Output & Persistence
User I/O → File I/O → JSON → Database

P2	`input()` / `print()`, f-strings
P3	text vs binary: `str` ↔ `bytes` (encode / decode)
P4	`with open(…) as f:` read / write / append
P6	`json` & `os` modules
P7–8	persist objects in a database (sqlite3)

TRACK C Capstone Project
One project grows every phase — e.g. a **Media Manager** or **Task Tracker**.

P1–2	pick the idea; name the data
P3	model records with list / dict
P4	menu loop with match / case
P5–6	split into functions & modules
P7	refactor into classes
P8	expose it as a FastAPI service

PROJECT LAYOUT

```
media_manager/
├ app/
│  ├ __init__.py
│  ├ models.py
│  ├ storage.py
│  └ api.py
├ tests/
│  ├ test_models.py
│  └ test_api.py
├ data/library.json
├ main.py
└ requirements.txt
```

DEFINITION OF DONE

- runs from `main.py` without crashing
- bad input handled with `try/except`
- data survives a restart (file / DB)
- one class per real-world thing
- every function has a unit test
- API returns correct status codes
- PEP 8 names & docstrings
- dependencies in `requirements.txt`

File I/O + OOP + Error handling + Tests = **Production-ready**

Day 1 · Day 2 · Covered in parallel · Mutable · Immutable